Final Project

Team registration due date: 02/10 (Tue)
Abstract (5%) due date: 03/03 (Tue)
Midpoint report (5%) due date: 04/09 (Thu)
Presentation (10%) due date: 04/28 and 04/30
Final report (20%) due date: 05/07 (Thu)

Project
- Should be done in teams of 3
- Exceptions (solo, duo, 4, 5) are permitted upon request with the form.

There're four-steps:

1. Team registration, or a request for a team match (together with team registration/request for a team match for A3) – Due 02/10 (Tue)
    a. We announce team matching by 02/17 (Tue)
2. (Checkpoint 1) Abstract submission (<250 words): Students should think about the topic in advance, and check if the topic is out of the scope – Due 03/03 (Tue) We'll also use this information for assigning breakout rooms and graders.
3. (Checkpoint 2) Midpoint project report (2–3 pages), requiring literature review, training data (if applicable), evaluation data and metrics, baselines, and preliminary results (such as evaluation of baselines). Due 04/09 (Thu). Brief feedback provided by 04/16 (Thu).
4. (Checkpoint 3) Project presentations (04/28 and 04/30) – likely breakout rooms; duration will be finalized based on # of teams
5. (Checkpoint 4) Project report (6–8 pages) – Due 05/07 (Thu)

Overview

The final project is an open-ended research or systems project in Natural Language Processing (NLP) or its intersections with areas like vision, health, HCI, education, security, information retrieval, agentic systems, or social science. Students may (a) design a project from scratch, (b) start from one of the example topics, or (c) modify/extend an existing idea, so long as the project has a clearly defined task and is evaluated with a reproducible experimental protocol.

A successful project should at least match the standard of a mini conference workshop paper: it states a well-scoped problem, motivates why it matters, proposes a method or analysis, and provides evidence through experiments and evaluation. Projects can be model-building, post-training, dataset/benchmark creation, evaluation/analysis tooling, or interactive systems. But they must include standard deliverables which will be mentioned below.

We strongly encourage choosing a scope that can be completed within the course timeframe. Students are free to extend their project after the final deadline and submit to peer review venues. Submit the pdf via gradescope. For the final report, you can submit it along with your public repository or add this email zn.tang.terran@gmail.com to your private repository.

We will later release details on compute resources.

General Project Deliverables

Students are free to design their own projects, simply reproduce an existing paper, or use/start-from/modify the example projects. Still, there are generally deliverables that apply to most project designs. For checkpoints 1 and 2, note that there are required deliverables. **Still, the content can be significantly changed for the final report. Email me if you are considering changing to a completely new topic.** If you need other special accommodations, please email GSI **Zineng Tang, terran@berkeley.edu**.

Checkpoint 1 deliverable:

1) Problem Definition + Scope

Checkpoint 2 deliverable:

2) Dataset / Environment Artifact

1. You can choose
    - New Dataset (collected)
    - Synthetic Dataset (procedurally generated)
    - (Subset of) Existing dataset

3) Baselines (at least 1-2)

1. Zero-shot / prompt-only baseline
2. Heuristic baseline
3. Existing SOTA model or open LLM baseline

4) Literature Review

5) Evaluation Suite + Metrics

1. Primary metric aligned to the task for your baseline (like accuracy)
2. Efficiency (optional)
3. Generalizability (optional)

Final deliverable:

6) Proposed Method / System (1 of 2)

1. Proposed method (could be a model, training paradigm, data generator, etc.)
2. Proposed analysis tool

7) Evaluation Suite + Metrics

4. Primary metric aligned to the task (like accuracy)
5. Efficiency (optional)
6. Generalizability (optional)

8) Ablations (at least 2)

1. Example:
    a. Remove a key component in your framework
    b. Compare different prompts

9) Error Analysis (optional)

1. Example
    - Insight: “If we fixed X, performance would likely improve”
    - Failure modes or examples

10) Qualitative Demonstrations

1. Example
    - A sample data if using synthetic dataset
    - Example model input or output

Writeup Requirements

Here is a general writeup structure for a 6-8 page 1 column paper, where references and acknowledgements are not counted. Use this package for either latex or .docx writing: https://github.com/ICLR/Master-Template/raw/master/iclr2026.zip. If you plan to use another template or use your own, please notify me to check if they are in good standards. The writeup quality should at least match with standard conference workshop submission.
Below are general guidelines for paper structure but not fully restricted. For example, you can reorganize your approaches into different method sections. If you are considering certain sections to be longer than usual or exceeding the paper page maximum, please email TA **Zineng Tang, terran@berkeley.edu**.
Note: Deliverables for different checkpoints and final reports should be accumulated, meaning that your latest writeup should include all previous checkpoints.

Checkpoint 1 deliverable:

1) Title + Abstract (0.5 to 0.8 page)

- Problem → method/approach → (anticipated) key results → why it matters

---

Checkpoint 2 deliverable:

2) Introduction (within 1 page)

3) Related Work (within 1 page)

- Literature review with references included

4) Baseline Setup (within 1 pages, can latter be merged with “6) Method Setup” and “7) Experiment”)

- Implementation details of baseline
- Dataset details or environment setup
- Evaluation setup and baseline results

5) If you would like to be considered for the VESSL AI credits, write a short paragraph on "Application to VESSL AI credits" describing your plans on how to use GPU hours.

Final deliverable:

6) Method Setup (1-3 pages)

- Core idea
- Implementation details (enough to replicate)
- Any theoretical framing

7) Experiment (1-2 pages)

- Hyperparameters, etc.
- Baselines and your method
- Main results table

8) Analysis (within 1 page)

- Ablations
- Error analysis and qualitative examples

9) Limitations and Broader Impact and Ethics if relevant (1-2 paragraphs, optional)

10) Conclusion (1 paragraph)

11) Appendices (optional)

- Additional experiments details like prompts
- Additional qualitative or quantitative results

Reference Paper Topics

Overview: We provide example paper topics for reference covering different NLP and Vision-Language areas. If you intend to select any of the below topics or modify them, feel free to email TA **Zineng Tang, terran@berkeley.edu** for any questions you have.
Fill out this google excel, https://docs.google.com/spreadsheets/d/1Bhi5m-IyiQ_49CVsYbA_nwNVFJgCfkY__3PCKVKGN0U/edit?usp=sharing with your name/email in the corresponding topic, if you are interested in any of the below topics but couldn't find a team.

Topic 1: Representing Language as World States in Symbolic Text Environments

World models have emerged as a powerful tool for representations in agents. However, language does not explicitly give underlying states. Easily, one could use a tool-calling or web-browsing environment to transform LLM into a flexible agent simulating a world model behavior. However, how do world states work in natural text? For the sentence, “I go to Starbucks and then buy a cappuccino.” It can be defined as

1. action_0: go to starbucks, state_0: I’m currently not at a Starbucks
2. action_1: buy a cappuccino, state_1: I’m currently at Starbucks but don’t have any coffee

Deliverables:

1. A well-defined action space, observation space, and states.
2. A well-defined task at which the “LLM world model” outperforms “just LLM”

Topic 2: Self-Play LLMs with Evolution Strategies

Post training with RL scales well but requires a lot of data. Recent works have shown that evolution algorithms have promising potential in optimizing data constrained tasks. Intuitively, well-designed multi-LLMs can operate more efficiently on certain tasks. Using the same copies of LLMs can enable more flexible communication. But there are many perspectives to go from there. For example, how multi-agents LLM prompts improve themselves through interaction. For multi-agents LLM to design an entire code project, how we can efficiently design their roles, e.g., verifiers, unit-tests writers, engine writers, etc. If we initialize with no role, can we design an evolution strategy that iterates prompts so that roles or better prompts can emerge?
Deliverables

1. A well-defined task showing how self-play with asymmetric roles could be better than plain copies of the same LLMs using standard post-training. The difference might be more significant in data-constraint environments.
2. An evolution strategy that well optimizes the prompts or training.
3. Prompt engineering that works well throughout tasks. Or, trained LLMs if you seek to improve LLM in multi-agents settings.

Topic 3: LLM as a pragmatic game player (RSA-style) in a controlled "Mini-World"
Build a reference game: speaker chooses description, listener chooses target among distractors. Implement Rational Speech Acts (RSA) with LLM likelihoods as components. An example toy setting would be: each scene is a set of objects with discrete attributes (e.g., color/shape/size/material/location) plus optional relations (left-of, inside, same-color-as). Speaker and listener are treated as policies over utterances. "LLM-RSA" uses the LLM as a scoring function: (i) as a literal listener, score how likely each candidate is given the utterance + scene description; (ii) as a literal speaker, score how likely an utterance is given a target + scene.
Deliverables:

1. A fully specified reference game environment:
    a. scene generator (objects + attributes + distractors + difficulty knobs)
    b. speaker/listener interfaces
    c. train/dev/test splits
2. A set of baselines:
    a. direct LLM speaker, direct LLM listener
    b. heuristic RSA (string-match semantics) vs LLM-RSA
    c. ablations on cost functions (length, rarity, ambiguity penalty).

Topic 4: Interactive error correction for summaries
A summarizer that asks one clarifying question when uncertainty is high ("Do you mean X or Y?"). Use uncertainty signals (entropy, self-consistency). For example, you can frame this as a two-stage pipeline: first, generate a provisional summary plus an internal "uncertainty report" that identifies which entities, times, causal links, or pronoun references are unstable under small prompt perturbations. Then decide: either finalize immediately, or ask exactly one clarifying question designed to maximally reduce downstream summary error. The clarifying question

should be constrained to a multiple-choice or binary format and should point to the exact ambiguity source (e.g., two plausible referents, two plausible time ranges, two competing interpretations of “it,” “they,” “the report,” etc.).
Deliverables:

1. A dataset of ambiguity-sensitive summarization prompts
2. A summarization model/pipeline that can either summarize immediately, or ask exactly one clarifying question when uncertainty is high.
3. Uncertainty signal implementation + ablations
4. Decision policy for when to ask like thresholding + calibration.

Topic 5: Memory illusions in LLMs
If you repeatedly suggest a wrong “fact” in conversation, does the model become more likely to endorse it later? Compare with and without retrieval grounding. One way to approach this is to use a controlled multi-turn persuasion setup. You create conversations where a user repeatedly injects the same false claim with increasing social pressure. You then probe later with questions that (a) directly ask for the facts, (b) ask indirectly, and (c) ask for justification or uncertainty (“how sure are you?”).
Deliverables:

1. A controlled conversation protocol that injects repeated false suggestions
2. A benchmark set of probe questions
3. Comparisons
    a. with vs without retrieval grounding
    b. with vs without “memory” features
    c. different prompt policies
4. Metrics: like endorsement rate, correction rate, and uncertainty rate.

Topic 6: OCR-free document understanding for messy notes
Use a VLM to extract key-value pairs from handwritten or messy forms without explicit OCR (or compare to OCR+LLM). The OCR-free pipeline uses a vision-language model that outputs a strict JSON object directly from the document entities.
Deliverables:

1. A dataset of messy documents with ground-truth key–value pairs.
2. Two Pipelines:
    a. VLM direct structured extraction
    b. OCR + LLM baseline

Topic 7: Counterfactual preference tuning
Instead of preference pairs “A vs B,” generate counterfactual pairs where only one latent factor changes: verbosity, hedging, epistemics, etc. Concretely, build a pair generator that starts from a base prompt + a reference answer, then produces minimal edits that isolate one control dimension while holding content constant.
Deliverables:

1. A generator that produces counterfactual preference pairs.
2. A compact control taxonomy + specification
3. Training runs for at least:
    a. a baseline post-trained model
    b. your counterfactual preference tuned model (e.g., DPO/IPO/GRPO variant).

Topic 8: Latent scratchpad without revealing chain-of-thought
Make a model that uses a latent reasoning channel (e.g., hidden states / discrete latent tokens) to improve performance, while only emitting short explanations. Concretely, during training, give the model access to a richer internal workspace than what it's allowed to show at inference time. There are a few practical implementations: (1) discrete latent tokens that the model generates but are stripped before user display, (2) an auxiliary reasoning decoder that produces hidden rationales used only for training signals, or (3) teacher–student distillation where the teacher produces full CoT and the student learns to match answers while generating only a short justification template.
Deliverables:
1. A pipeline implementing latent reasoning.
2. A training recipe that improves performance while keeping visible explanations short.
3. Baselines:
    a. direct model with visible CoT
    b. direct model without CoT
    c. your latent-scratchpad model

Topic 9: Screenshots-to-Simulation
Given an image of a website UI or game, can you generate code enough to simulate it. Define simulation as: the generated artifact renders to a visually and structurally similar UI, and supports a minimal set of interactions (click, scroll, input) that behave like the reference.
Deliverables:
1. A task and evaluation metric that can evaluate the accuracy of generated code or rendered image.
2. A trained model or reproducible prompt engineering of an LLM/VLM.
3. A results report and demo showing the rendered image/video outside of your training data.

Topic 10: Pre-training data for Time-locked LLMs
Overview: History LLM has proposed the development of time-locked LLMs, whose training data is strictly restricted to text created prior to a specific cutoff date (e.g., 1910). These models are intended to function as windows into the past, enabling historically grounded research across the humanities, social sciences, and computer science. We refer readers to the HistoryLLM repo for further motivation, including a discussion on why prompting GPT-5 to role-play is insufficient.

HistoryLLM does not release their training data. The goal of this project is to collect, curate, and release a high-quality pre-training corpus consisting exclusively of text written before a target

timestamp. You will apply systematic data curation procedures, such as linearization, filtering and deduplication, to construct a dataset that is both sufficiently large and high quality.
Deliverables:

1. A pre-training corpus composed of text written prior to a specific time cutoff, with sufficient scale and quality that is readily available to train a language model.
2. Quantitative metrics for evaluating pre-training data quality, along with ablation studies examining the impact of different data curation choices based on these metrics.
3. Qualitative analyses of the resulting corpus.
4. (If compute allows, or if selected for VESSL AI credits) Train a modest-size language model on the data.

Topic 11: Evaluation of Time-locked LLMs

History LLM has proposed the development of time-locked LLMs, whose training data is strictly restricted to text created prior to a specific cutoff date (e.g., 1910). These models are intended to function as windows into the past, enabling historically grounded research across the humanities, social sciences, and computer science. We refer readers to the HistoryLLM repo for further motivation, including a discussion on why prompting GPT-5 to role-play is insufficient.

However, HistoryLLM does not go through a systematic evaluation of the resulting models. The goal of this project is to develop a comprehensive evaluation suite tailored to HistoryLLM-style time-locked models. The suite will include a diverse set of tasks that such models are expected—or expected *not*—to perform. Such tasks can include:

1. Standard language understanding tasks, generation tasks, and even knowledge tasks that do not require knowledge beyond the temporal cutoff;
2. Tasks designed to probe capabilities that HistoryLLM is intended to support, such as predicting future events that are deterministically implied by historical context; and
3. Tasks that a properly time-locked model should fail, such as tasks that must not be solved without information after the cutoff date.

Topic 12: Rethinking Contamination

Overview: Benchmark contamination—when training data unintentionally or intentionally includes evaluation benchmarks—is a well-known issue. Current decontamination techniques largely rely on n-gram overlap (e.g., excluding a training document if it shares more than x% of y-gram overlap with a test question). These methods are loose and prone to false negatives. This project aims to revisit contamination in popular training datasets using a more accurate, retrieval + LLM-based approach for detecting near-duplicate content.
Deliverables:

1. Consider three pairs of <train dataset, eval benchmarks>: (1) <Tulu 3's training data, their benchmarks>, (2) <s1's training data, their benchmarks>, (3) <OpenThoughts' training data, their benchmarks>. Read the corresponding papers and summarize how each work attempted to decontaminate its training data.
2. Implement a stronger decontamination pipeline that (1) first retrieve the top-K training samples relevant to each test question (using BM25 or neural embeddings), and (2) use

an LLM (e.g., Llama-3.3-70B-Instruct or GPT-5) to classify whether each retrieved sample is a near-duplicate.

3. Measure the extent of benchmark contamination using this improved method. When contamination is detected, analyze the likely causes: (a) the original work did not perform decontamination; (b) the original work claimed to perform decontamination, but the procedure was ineffective or incorrectly applied; or (c) the original work performed decontamination correctly, but the method itself was insufficiently strong.
4. Finally, extend the analysis to additional training datasets and benchmarks, and extract broader insights into the effectiveness and limitations of existing decontamination practices.

Topic 13: Rethinking Retrieval Evaluation

Overview: Traditional retrieval evaluation relies on annotated positives and metrics that check whether those positives appear in the top-K retrieved items. This approach suffers when annotators miss relevant documents (false negatives), unfairly penalizing the retrieval model. An LLM-based post-hoc judge (evaluating each retrieved document for true relevance) may provide a more accurate assessment. The goal is to revisit popular retrieval benchmarks and re-evaluate models using LLM-based judging.

Deliverables:

1. Collect retrieval benchmarks (e.g., BEIR, BRIGHT, MTEB) and strong retrieval models (selected from leaderboards). Obtain retrieval predictions either from released artifacts or by running the models locally (after ensuring that reported evaluation results are reproducible).
2. Implement an LLM-based evaluation method to check relevance of each top-K retrieved document (e.g., Llama-3.3-70B-Instruct or GPT-5 as a judge). Analyze: (1) How many false negatives are uncovered? (2) Do rankings or performance trends of models change?
3. Extend the analysis to additional benchmarks and retrieval models, and synthesize broader insights into the reliability and limitations of existing retrieval evaluation protocols.

Topic 14: Data valuation

Determining the value of data for training an LLM is surprisingly hard. Data value depends not only on intrinsic properties—such as size and quality—but also on how it complements the data a model already has, making the impact of new data difficult to predict. This project frames data valuation as a scaling-law problem. Specifically, given a large existing dataset D and a tiny uniformly sampled subset from a candidate dataset D_new, this project aims to study how to estimate the expected performance impact of acquiring the full D_new, assuming its total size is known.

Deliverables

1. A well-defined problem formulation and experimental setup spanning a diverse set of <D, D_new, evaluation benchmarks> configurations, along with appropriate quantitative metrics.
2. Proposals of multiple viable methods for estimating data value under this formulation.

3. Empirical comparisons and ablation studies of the proposed methods, accompanied by analysis and broader insights into their applicability and limitations.

Topic 15: Quantifying the impact of "Consent-in-Crisis"

The Consent in Crisis paper showed a significant decline in the availability of high-quality web data for model training; nearly 50% of previously accessible sources are now restricted, with removed data concentrated in known-to-be-high-quality sources such as news, encyclopedias, forums, and academic content. This raises serious concerns about the long-term quality and diversity of publicly available training datasets. However, the actual impact of this content shrinkage on model performance has not yet been quantified. Part of a challenge is that data-availability labels exist only for a tiny fraction of the data, making it difficult to estimate availability for the full corpus. This project aims to (1) infer availability at scale, and (2) measure the downstream performance impact of training on a "post-crisis" dataset.

Deliverables:

1. Propose methods for inferring data availability at scale when explicit consent information is unavailable. Design experimental setups to empirically validate these methods using quantitative metrics, and conduct ablation studies to assess their reliability.
2. Quantify the impact of dataset contraction on model performance. Ideally, this involves training two models: one on a "pre-crisis" dataset and another on a "current-crisis" dataset. If computational resources are limited, develop alternative evaluation strategies to estimate the performance impact.
3. Optionally, extrapolate observed trends to predict future content availability and train a "future-crisis" model to assess the longer-term consequences of continued data shrinkage.

Topic 16: Slide QA Benchmark and VLM Agent for Course Lecture Slides

Overview: Can we build a question-answering benchmark for open-domain questions about CS 288 and develop an agent that can retrieve the relevant lecture slide, reason over it, and produce an accurate answer? Since lecture slides often contain rich visual information, purely text-based retrieval is insufficient. Both the retrieval and generation components should therefore be based on vision-language models (VLMs).

Deliverables

- A QA benchmark built from three or more publicly available courses, including CS 288—either multiple Berkeley courses or NLP-related courses from different universities. The benchmark should feature a diverse set of question types, including: (1) questions answerable by reading text alone, (2) questions requiring interpretation of images, (3) questions involving tables, (4) questions involving charts or graphs, and (5) questions requiring understanding of complex slide layouts. The benchmark should be accompanied by a reliable and well-defined evaluation metric.
- A VLM-based agent capable of retrieving the most relevant slides and answering questions based on their content. The system should include a comprehensive set of ablation studies comparing different design choices.
- A publicly available, web-based demo showcasing the agent's capabilities.

Topic 17: ToS-Respecting Web Slicing for Responsible AI Training
Context: Modern AI models rely heavily on large-scale web data, but much of this data exists in a legal and ethical gray zone. Even widely used sources such as Common Crawl or proprietary web datasets may not be safe for training due to site-specific Terms of Service (ToS). Today, there is no standardized, machine-readable way to understand or enforce these terms beyond coarse signals like robots.txt. As a result, ToS information is fragmented, inconsistent, and largely ignored in practice.

The Consent in Crisis paper demonstrated that ToS constraints matter and can be systematically analyzed—but only through extensive manual effort on a tiny subset of the web. Scaling this approach remains an open and urgent challenge. Without automation, responsible data use simply does not scale.

This project asks: Can we turn the web's messy, unstructured ToS landscape into an actionable, machine-interpretable layer that enables responsible data use at scale? Project Goal The goal is to automatically extract, organize, and operationalize website Terms of Service, enabling AI developers—and data users more broadly—to slice large web datasets according to permitted uses. Instead of treating web data as "all or nothing," this project enables purpose-aware, ToS-compliant dataset construction.

Deliverables
- Automated ToS Discovery Agent: A web agent that, given a URL, automatically discovers and extracts its Terms of Service in unstructured form. The system should handle the diversity and inconsistency of ToS documents across the web. (This does not need to be LLM-based.)
- ToS Database for Curated Web Corpora: Using the agent, build a large-scale database of ToS metadata for a high-quality subset of Common Crawl, such as DCLM or FineWeb-Edu.
- Purpose-Aware Data Slicing Model: A model that takes a stated data usage purpose (e.g., commercial training, academic research, redistribution) and automatically returns the subset of data whose ToS permits that use.

Topic 18: Email management
How well can we use LLMs to automatically process an email inbox, for example by re-sorting the inbox to show the highest-urgency emails? This is not simply a matter of classifying each email as urgent or not urgent: each email might be missing a lot of context, and the users of the system may have very different preferences from one another.

Deliverables: a system that takes as input an email inbox (e.g., .mbox) and:
- Processes emails:
    - Processes each email independently using LLMs to assign it some urgency score (e.g., "due date for response")

    - Efficiently processes all emails together (e.g., via clustering) to refine urgency scores
    - Identifies when context might be missing that is necessary to assign a more accurate urgency score
- Learns from the user:
    - An interface that chooses the most “uncertain” labels to have the user provide a correction, and/or additional context in the form of linguistic feedback
    - A way to incorporate the user’s feedback into the system so that it results in better performance (e.g., through RLHF, automatic prompt engineering, or history management/compression)
- Works locally: for privacy reasons, it would be great if such a system can work locally, with small LLMs that can run in-browser or on a PC (no GPUs). What challenges come up here that wouldn’t if we can assume access to APIs or large amounts of GPUs? How does this change the design of the system?

Topic 19: Digital tutors
(V)LMs are getting increasingly better at joint multimodal processing of video, audio, images, and text. Can it teach you a new skill in the real world? Here, you will create and benchmark a system based on VLMs that teaches users a new skill, for example, how to use a complex piece of software like Figma or Blender.
Deliverables:
- A system that processes streaming inputs from the user (audio, video) and generates a stream of outputs (audio). The system should be able to engage in a conversation with its user as the user attempts to learn a new skill.
- A set of evaluation criteria how to benchmark such a model at scale. What kinds of behaviors are important for it to exhibit as a tutor, and which ones do the models still have room to improve? Is it feasible to do automatic evaluation, and if so, how would we build a user model?
- Experiments with limiting the modalities of the system: what if we replace audio with text? What if we lower the framerate on the video so that we lower the model memory requirements?

Topic 20: Spatial language
VLMs are notoriously bad at understanding referring expressions, especially ones that require perspective-taking and spatial reasoning (see: https://aclanthology.org/2024.emnlp-main.1100/). The aim of this project is to (a) characterize the use of spatial language in existing vision-language datasets (e.g. DataComp, caption datasets), including what kinds of spatial relations exist and how they are grounded in the image, and (b) train models to be better at generating and understanding spatial relations through fine-tuning models with synthetically-generated data.

Topic 21: Evolutionary Strategies for Gradient-Free Model Training
Reference Papers:
1. https://arxiv.org/abs/2601.20861

2. https://arxiv.org/abs/2509.24372

One of the biggest missing capabilities in current AI systems is the ability to learn continuously after deployment. Implementing such continually learning systems has several challenges, one of which is the large memory requirement of gradient-based algorithms that are used to train state-of-the-art LLMs. Evolutionary Strategies (ES) have recently re-emerged as a gradient-free alternative to traditional learning algorithms and have shown encouraging performance on specific tasks in LLMs. However, we've recently shown that using ES to train LLM leads to significant catastrophic forgetting.

The aim of this project will be to mitigate forgetting issues when training using ES.

Topic 22: Learning Continually "during" Problem Solving

The focus here is on long-context tasks, where it is impossible to finish a task within the context length of a model. An example problem statement could be research ideation, where the model has to read 50-100 papers before it comes up with novel ideas. Fitting that many papers clearly go beyond the context length of the model. It is possible to do this with context management, but we want to do this by ingesting model experience into the model weights. This is a "long time-horizon sparse reward" problem statement setting.

The aim of this project will be to come up with algorithms to enable in-weight experience ingestion intermittently during the problem solving process.